INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO RIO GRANDE DO NORTE
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
SISTEMAS OPERACIONAIS · 2026

AULA 05 · LABORATÓRIO 2

Os limites do processo: heap e stack

Onde cada variável mora, quanto o processo realmente ocupa e o que acontece quando ele passa do limite

`VmSize ≠ VmRSS` · `malloc → NULL` · `recursão → SIGSEGV` · `ulimit → o teto`

UNIDADE	2 — Gerência de processos
TEMA	Espaço de endereçamento: text, data, bss, heap e stack
LINGUAGEM	C, mais `/proc/<pid>/maps`, `status` e `ulimit`
AMBIENTE	Container `so-ifrn` da disciplina — limites controlados
ENTREGA	Previsões escritas + relatório PDF + defesa oral
REFERÊNCIA	Maziero, *Sistemas Operacionais: Conceitos e Mecanismos*, §15.2, p. 186-187; OSTEP, cap. 13

ROTEIRO DO LABORATÓRIO

1 Onde cada variável mora — endereços conferidos no `maps`
2 O heap cresce — reservar endereço não é usar memória
3 O limite do heap — `malloc()` devolvendo `NULL` sob `ulimit -v`
4 A pilha cresce — recursão até o `SIGSEGV`
5 Mudando o limite da pilha com `ulimit -s`
6 Arquivo grande exige muita RAM? — as duas estratégias
7 Comparação final · síntese, questões e entregáveis

A pergunta central

> *Até onde um processo consegue crescer?*

O Laboratório 1 mostrou o processo **no tempo** — nascendo, bloqueando, terminando. Este mostra o processo **no espaço**: quanto de memória ele ocupa, onde cada coisa mora, e o que acontece quando ele pede mais do que pode ter.

Objetivos

Ao final você será capaz de:

1. localizar **heap** e **stack** no mapa de memória de um processo real;
2. explicar a diferença entre **VmSize** e **VmRSS**, e por que ela é enorme;
3. mostrar que `malloc()` **reservar** memória não é o mesmo que **usar** memória;
4. provocar e explicar a falha do heap e a falha da pilha — e dizer por que são diferentes;
5. relacionar o limite da pilha com a profundidade máxima de recursão;
6. decidir, num problema real, se processar um arquivo grande exige muita RAM.

Conhecimentos prévios

C básico (`malloc`, ponteiros, recursão), terminal Linux, e a noção de espaço de endereçamento apresentada na aula.

Ambiente

```
./so.sh                # macOS e Linux
.\so.ps1               # Windows (PowerShell)
```

```
cd "/so/Aula 05/codigo/lab2"
gcc -Wall -Wextra -O0 -o mapa    mapa.c
gcc -Wall -Wextra -O0 -o heap    heap.c
gcc -Wall -Wextra -O0 -o pilha   pilha.c
gcc -Wall -Wextra -O0 -o arquivo arquivo.c
```

> O `pilha.c` compila com um aviso: `warning: infinite recursion detected`. **É esperado** — o compilador está certo, a recursão é infinita de propósito. Guarde esse aviso: ele volta numa das questões finais.

O `-O0` não é decoração

Com otimização, o compilador pode transformar a recursão do `pilha.c` em laço (*tail call*) e o programa **nunca** estouraria a pilha. Compile sempre com `-O0` nesta prática.

Parte 1 — Onde cada variável mora

> **PREVEJA ANTES DE SEGUIR**
>
> Um programa em C tem: uma variável global inicializada, uma global zerada, uma local e um bloco de `malloc()`. **Escreva sua hipótese sobre quais delas ficam perto umas das outras** na memória, e quais ficam longe.

Execução

Terminal 1:

```
./mapa &
```

O que observar

```
PID = 24

  o que é                       endereço   região esperada
  ------------------------------------------------------------------
  código (função main)     0xaaaacb360b10   text  — só leitura + execução
  string literal           0xaaaacb360d28   rodata — dentro da área de código
  global inicializada      0xaaaacb380010   data
  global zerada            0xaaaacb380024   bss
  bloco de malloc(4096)    0xaaab0a5102a0   [heap]
  variável local           0xffffc912f9ec   [stack]
```

Agora, no terminal 2, com o PID que apareceu:

```
cat /proc/24/maps
```

```
aaaacb360000-aaaacb362000 r-xp  /tmp/mapa      ← text
aaaacb37f000-aaaacb380000 r--p  /tmp/mapa      ← rodata
aaaacb380000-aaaacb381000 rw-p  /tmp/mapa      ← data + bss
aaab0a510000-aaab0a531000 rw-p  [heap]
ffffc910f000-ffffc9130000 rw-p  [stack]
```

Interpretação

Confira endereço por endereço. Cada um cai dentro de uma faixa:

variável	endereço	faixa do `maps`	✓
`main`	`aaaacb360b10`	`aaaacb360000–aaaacb362000`	text
global inicializada	`aaaacb380010`	`aaaacb380000–aaaacb381000`	data
`malloc(4096)`	`aaab0a5102a0`	`aaab0a510000–aaab0a531000`	**[heap]**
local	`ffffc912f9ec`	`ffffc910f000–ffffc9130000`	**[stack]**

Repare na **distância**: o heap está em `0xaaab…` e a pilha em `0xffff…`. São regiões separadas por um vão gigantesco de endereços não mapeados. **Esse vão é de propósito** — é o orçamento comum que permite às duas crescerem em direções opostas sem decidir de antemão quanto cada uma vai precisar.

Repare também na coluna de permissões: `r–xp` no código (legível e executável, **não gravável**) e `rw–p` nos dados. É a proteção acontecendo.

> 💡 O `malloc(4096)` deste programa caiu dentro do `[heap]` porque é um bloco **pequeno**: a biblioteca C pede esse espaço ao núcleo crescendo uma região contígua (`brk`), que é o `[heap]`. Um bloco **grande** (acima de um limiar que a própria biblioteca decide, tipicamente na casa dos 128 KiB) costuma vir de um `mmap()` avulso, fora do `[heap]` — você vai ver isso de relance na Parte 2, quando o `heap.c` pedir blocos de 1 MiB. A relação real é "`malloc()` pede memória ao núcleo", não "`malloc()` é o `[heap]`".

Os quatro números do `status`

```
grep -E 'VmSize|VmRSS|VmData|VmStk' /proc/24/status
```

```
VmSize:     2260 kB
VmRSS:      1460 kB
VmData:      208 kB
VmStk:       132 kB
```

campo	o que é
`VmSize`	**tamanho virtual**: todo o endereço que o processo tem reservado
`VmRSS`	*resident set size* — a memória **física** de fato ocupada agora
`VmData`	memória privada que o núcleo classifica como "dados" do processo — inclui o heap, mas não só ele
`VmStk`	o tamanho atual da pilha

Já aqui `VmSize` (2260) é maior que `VmRSS` (1460). Guarde essa diferença: a Parte 2 vai transformá-la em abismo.

Encerre com `kill 24` (ou espere: o programa sai sozinho em 300 s).

Parte 2 — O heap cresce

> **PREVEJA ANTES DE SEGUIR**
>
> > *Um programa chama `malloc(1 MiB)` **quatrocentas vezes** e nunca escreve nos blocos. Quanta memória **física** você acha que o processo vai estar usando no fim?*

Experimento A — reservar sem tocar

```
./heap 400 nao
```

```
   MiB    VmSize(kB)     VmRSS(kB)    VmData(kB)
   384        397012          2924        394960
   400        413460          2988        411408
```

Interpretação

400 MiB reservados. 3 MB usados.

`VmSize` foi a 413 MB, mas `VmRSS` ficou em 2988 kB. O `malloc()` devolveu endereços válidos para todos os 400 blocos, e o processo pode usá-los quando quiser — mas **nenhuma página de memória física foi entregue**, porque o programa nunca escreveu neles.

> 💡 Reservar endereço é barato; usar memória é que custa. É por isso que `VmSize` sozinho não diz quase nada sobre consumo real de um processo — e por que `top` mostra as duas colunas. Como essa entrega sob demanda funciona por dentro é assunto da Unidade 3.

Experimento B — tocar a memória

```
./heap 400
```

Agora o programa escreve em cada bloco (`memset`). Acompanhe no terminal 2:

```
watch -n 1 'grep -E "VmSize|VmRSS|VmData" /proc/$(pgrep -n heap)/status'
```

Medido com `./heap 120`:

```
   MiB    VmSize(kB)     VmRSS(kB)    VmData(kB)
    80         84500         83636         82448
    96        100948        100084         98896
   112        117396        116532        115344
```

Agora `VmRSS` acompanha `VmSize` quase linha a linha — 117396 contra 116532. Cada `memset` força a entrega de páginas físicas de verdade. Compare com o experimento A, onde a segunda coluna ficou parada em 3 MB enquanto a primeira ia a 413 MB.

Parte 3 — O limite do heap

> **PREVEJA ANTES DE SEGUIR**
>
> > *O que acontece quando o processo pede mais memória do que pode ter: o `malloc()` devolve `NULL`, ou o programa é morto? Escreva sua hipótese.*

Cuidado de segurança

⚠️ **Não rode este experimento sem limite.** O objetivo é *observar* um limite, não travar o computador. Usamos `ulimit -v`, que limita **só este processo** e não afeta o resto da máquina.

Execução

```
( ulimit -v 262144 ; ./heap 2048 ; echo "código de saída: $?" )
```

Os parênteses importam: eles criam um subshell, então o `ulimit` vale só ali dentro. `262144` kB = **256 MiB**.

O que observar

```
   240        248980        248120        246928

>>> malloc() FALHOU ao pedir o MiB nº 253
>>> o processo continua vivo e pôde tratar o erro.
código de saída: 1
```

Interpretação

O `malloc()` devolveu `NULL` e o programa continuou vivo. Ele detectou a falha, imprimiu uma mensagem e saiu ordenadamente com código 1.

Este é o comportamento *desejável*: o pedido foi **recusado**, e o programa teve a chance de reagir. É por isso que **todo `malloc()` precisa ter o retorno testado** — e é exatamente o que o `heap.c` faz na linha ❷.

Repare que a falha veio no MiB nº 253, não no 256: o processo já tinha alguns MB de código, bibliotecas e pilha contando para o mesmo limite.

Variação — o limite do container

O container desta disciplina pode ser iniciado com um teto de memória:

```
docker run --rm --memory=512m -v "$PWD":/so -w "/so/Aula 05/codigo/lab2" so-ifrn bash
```

Dentro dele, `cat /sys/fs/cgroup/memory.max` mostra `536870912` — exatamente 512 MiB.

> **ATENÇÃO**
>
> ⚠ **Medido nesta imagem:** com o teto do container, o `./heap 2048` **não falhou nem foi morto**. O `VmRSS` estacionou perto dos 512 MB e o programa continuou rodando — o núcleo foi recuperando páginas antigas em vez de matar o processo, porque a VM do Docker Desktop tem área de troca. Em uma máquina sem essa folga, o desfecho seria o *OOM killer* encerrando o processo. **É por isso que o experimento oficial usa `ulimit -v`**: ele é determinístico e dá o mesmo resultado em qualquer máquina.

Isso é, por si, uma lição: **o mesmo programa tem desfechos diferentes conforme o ambiente**.

Parte 4 — A pilha cresce

> **PREVEJA ANTES DE SEGUIR**
>
> > *Cada chamada de função guarda seus dados na pilha. Se uma função chama a si mesma sem parar, **o que acontece primeiro**: a memória da máquina acaba, ou alguma outra coisa?*

Como a pilha cresce

Cada chamada empilha um **quadro** (*stack frame*) com o endereço de retorno, os parâmetros e as variáveis locais. No `pilha.c` a local é um `char buffer[1024]`, então cada quadro custa pouco mais de 1 KiB **neste programa, nesta compilação**. O tamanho real do quadro depende do código, do compilador, da arquitetura e da ABI — não existe um número fixo de "quanto custa uma chamada de função":

```
explode(1)     ← quadro: retorno + nivel + buffer[1024]
   ↓
explode(2)     ← mais um quadro
   ↓
explode(3)     ← mais um
   ↓
  ...          ← a pilha desce, e desce, e desce
   ↓
         ✗ limite atingido
```

Execução

```
ulimit -s
./pilha
echo "código de saída: $?"
```

O que observar

```
8192

nível     6000   pilha usada ≈    6280 KiB   quadro em 0xffffcb4ede08
nível     6500   pilha usada ≈    6803 KiB   quadro em 0xffffcb46b048
nível     7000   pilha usada ≈    7327 KiB   quadro em 0xffffcb3e8288
nível     7500   pilha usada ≈    7850 KiB   quadro em 0xffffcb3654c8
Segmentation fault
código de saída: 139
```

Interpretação

O limite da pilha chegou antes da memória da máquina. O `ulimit -s` diz `8192` KiB = 8 MiB, e o programa morreu tendo consumido **7850 KiB** — encostado no teto. O que falta para os 8192 é a margem ocupada pelos argumentos, pelo ambiente e pelo próprio quadro do `main`.

Repare nos endereços dos quadros: `…ff28`, `…4828`, `…9128`. **Estão diminuindo.** A pilha cresce para baixo, e cada quadro fica num endereço menor que o anterior.

O código de saída **139** segue a regra de sempre: **128 + 11 = SIGSEGV**. Confira com `kill -l 11`.

> Este programa morre **de propósito**. O objetivo não é "causar um erro": é ver que o limite existe, que ele é configurável, e que o sistema o faz valer.

Parte 5 — Mudando o limite da pilha

> **PREVEJA ANTES DE SEGUIR**
>
> *Se você **dobrar** o limite da pilha, o que acontece com a profundidade máxima da recursão?*

Execução

```
( ulimit -s 8192  ; ./pilha ; echo "saída: $?" )
( ulimit -s 16384 ; ./pilha ; echo "saída: $?" )
```

O que observar

`ulimit -s`	último nível alcançado	pilha usada	código de saída
8192 KiB (8 MiB)	**7.500**	7.850 KiB	139
16384 KiB (16 MiB)	**15.500**	16.225 KiB	139

(o programa imprime de 500 em 500, então o último nível impresso é o múltiplo de 500 imediatamente anterior à falha — a profundidade real é um pouco maior)

Interpretação

Dobrou o limite, dobrou a profundidade — 7.500 para 15.500, praticamente 2×. A relação é direta:

```
limite da pilha
        ↓
quantos quadros cabem
        ↓
profundidade máxima da recursão
```

E o desfecho é o mesmo nos dois casos: `SIGSEGV`, saída 139. **Aumentar o limite não conserta uma recursão infinita** — só adia a falha. A correção de verdade é no algoritmo.

> 💡 O `ulimit -s` vale para os processos criados **depois** dele, naquele shell. Por isso os parênteses: cada subshell tem o seu.

Parte 6 — Arquivo grande exige muita RAM?

> **PREVEJA ANTES DE SEGUIR**
>
> > *Seu programa precisa somar todos os bytes de um arquivo de **10 GB**. Ele precisa de 10 GB de RAM? Responda sim ou não **antes** de rodar.*

Preparação

```
dd if=/dev/urandom of=/tmp/g.bin bs=1M count=100 status=none
ls -lh /tmp/g.bin
```

Execução — duas estratégias, o mesmo resultado

```
./arquivo incremental /tmp/g.bin
./arquivo inteiro     /tmp/g.bin
```

O que observar

```
--- incremental ---
arquivo: /tmp/g.bin  (100.0 MiB)
soma de todos os bytes = 13368958236
VmRSS depois de ler:    1336 kB   <<< o número que interessa

--- inteiro ---
arquivo: /tmp/g.bin  (100.0 MiB)
soma de todos os bytes = 13368958236
VmRSS depois de ler:  103744 kB   <<< o número que interessa
```

Interpretação

A soma é idêntica: `13368958236`**.** Os dois programas fazem exatamente o mesmo trabalho e chegam exatamente ao mesmo resultado.

A memória não é. `1336 kB` contra `103744 kB` — uma diferença de **78×**.

A diferença está em duas linhas de código:

```
/* incremental: um balde de 4 KiB, reusado do começo ao fim */
char balde[4096];
while ((n = read(fd, balde, sizeof(balde))) > 0)
    /* processa os n bytes */ ;

/* inteiro: um bloco do tamanho do arquivo, tudo de uma vez */
char *tudo = malloc(tamanho_do_arquivo);
read(fd, tudo, tamanho_do_arquivo);
```

O arquivo pode ter 100 MiB ou 100 GiB — **o balde continua com 4 KiB**.

E quando a estratégia não cabe

```
( ulimit -v 131072 ; ./arquivo inteiro /tmp/g.bin ; echo "saída: $?" )
```

```
>>> malloc() FALHOU ao pedir 100.0 MiB de uma vez.
>>> este é o ponto do experimento: a estratégia não cabe.
saída: 1
```

Com 128 MiB de teto, a estratégia "carregar tudo" simplesmente **não é viável** — enquanto a incremental rodaria sem dificuldade.

A conclusão

```
arquivo grande ≠ necessariamente muita memória
```

O consumo depende da **estratégia do programa**, não do tamanho do arquivo. Programas que processam dados em fluxo (*streaming*) rodam com memória constante, independentemente do tamanho da entrada.

Limpe: `rm /tmp/g.bin`

Parte 7 — Comparação final

Preencha:

Característica	Heap	Stack
Principal finalidade		
Como cresce		
Quem pede a memória		
Exemplo de uso		
Quem libera		
Onde fica o limite		
Falha possível		
Como a falha se manifesta		

Perguntas de investigação

1. Na Parte 2, `VmSize` chegou a 413 MB com `VmRSS` em 3 MB. Explique a diferença sem usar a palavra "virtual" — descreva o que aconteceu, não o nome do mecanismo.
2. Na Parte 3 o `malloc()` falhou **devolvendo NULL**. Na Parte 4 o programa **foi morto** por um sinal. Por que os dois limites falham de formas tão diferentes?
3. O `pilha.c` compila com `warning: infinite recursion detected`. Se o compilador **detectou** o problema, por que ele não impede a compilação?
4. Em `mapa.c`, a global inicializada e a global zerada ficaram em endereços vizinhos (`…380010` e `…380024`), mas em seções diferentes (`data` e `bss`). Se estão coladas, por que o compilador as separa?
5. A pilha cresce para baixo e o heap para cima. O que aconteceria se os dois crescessem na mesma direção?
6. Você precisa processar um CSV de 40 GB numa máquina com 8 GB de RAM. Qual das duas estratégias da Parte 6 você usa, e qual é a limitação que a outra teria?

Cuidados de segurança

- **Sempre** limite o heap com `ulimit -v` num subshell antes de rodar o `heap.c` sem teto.
- O `heap.c` já tem trava interna de 8192 MiB, mas ela é a última defesa, não a primeira.
- O `pilha.c` é seguro: ele morre sozinho em segundos e não afeta nada fora do processo.
- Não crie arquivos de teste gigantes no `/so` — use `/tmp`, que fica dentro do container e some quando você sai.
- Nenhum experimento aqui precisa de `sudo`.

Desafio

O `heap.c` aloca blocos de 1 MiB e **nunca chama** `free()`. Modifique-o para liberar cada bloco logo depois de tocá-lo.

Entregue: o novo `VmRSS` no fim da execução, e a resposta: o `VmSize` voltou ao valor inicial? Se não voltou, formule uma hipótese sobre por quê — e diga como você a testaria.

Questões finais

1. Qual a diferença entre **heap** e **stack**?
2. Qual a diferença entre `VmSize` e `VmRSS`?
3. Qual a relação entre `malloc()` e o heap?
4. Qual a relação entre **chamadas de função** e a stack?
5. Por que recursão excessiva gera `SIGSEGV`, e não uma mensagem de "memória insuficiente"?
6. Por que um arquivo de 10 GB **não** exige necessariamente 10 GB de RAM?
7. Por que o sistema operacional precisa controlar os recursos dos processos? Responda citando pelo menos **dois** experimentos deste laboratório.

Atividade de síntese

Desenhe, à mão, o mapa do espaço de endereçamento do processo `mapa` **com os endereços que a sua máquina produziu**. Marque: `text`, `data` / `bss`, `[heap]`, o vão, e `[stack]`. Calcule, a partir do `maps`, o tamanho em KiB do heap e da pilha (subtraia os endereços em hexadecimal).

Abaixo do desenho, responda em até 5 linhas a pergunta central:

> *Até onde um processo consegue crescer?*

Critérios de entrega

#	item	o que precisa ter
1	as **previsões** 🔮 das Partes 1 a 6	escritas **antes** de rodar, com o desfecho real ao lado
2	a tabela endereço × faixa do `maps`	com **os seus** endereços
3	`VmSize` e `VmRSS` nos dois modos do `heap`	com a razão entre eles calculada
4	a saída do `malloc()` falhando	com o `ulimit -v` usado e o código de saída
5	a tabela de `ulimit -s` × profundidade	os dois limites, com os seus números
6	as duas execuções do `arquivo`	mostrando **a mesma soma** e RSS diferentes
7	a tabela comparativa da Parte 7	preenchida

#	item	o que precisa ter
8	respostas às 6 perguntas de investigação	
9	as 7 questões finais	
10	a atividade de síntese	desenho + cálculo + 5 linhas
11	*(opcional)* o desafio	código + medição + hipótese

Erros comuns

Sintoma	Causa	O que fazer
`./pilha` não morre	compilou com `-O2`	recompile com `-O0`; a otimização vira a recursão em laço
`ulimit -v` reclama de permissão	você tentou **aumentar** um limite já baixado	limites só descem; abra um shell novo
o `ulimit` não afeta o programa	você o rodou fora do subshell	use `( ulimit -v N ; ./prog )`, com os parênteses
`pgrep -n heap` não acha nada	o programa já terminou	aumente o teto: `./heap 2048`
o container ficou lento	você rodou o `heap` sem limite	saia e entre de novo; e leia *Cuidados de segurança*
`VmRSS` não sobe no `./heap`	você usou o modo `nao`	é o ponto do experimento A; rode sem o `nao` para o B

Arquivos deste laboratório

arquivo	o que é
`codigo/lab2/mapa.c`	imprime o endereço de cada região e pausa para inspeção
`codigo/lab2/heap.c`	aloca 1 MiB por vez, com e sem tocar a memória
`codigo/lab2/pilha.c`	recursão até estourar o limite da pilha
`codigo/lab2/arquivo.c`	duas estratégias de leitura, medindo o `VmRSS`
`LAB2-limites-heap-stack.md`	este roteiro
`LAB1-ciclo-de-vida.md`	o laboratório anterior
`LAB3-buffer-e-fluxo.md`	o laboratório seguinte

Todas as saídas deste roteiro foram medidas na imagem `so-ifrn` (Ubuntu 24.04.4, kernel 6.12.76-linuxkit aarch64, GCC 13.3.0) em 19/09/2026.